

Proiect Baze de Date: Examenale Naționale

Student: Tiloiu Darius-Lucian

Grupa: 144

1. Descrierea modelului real, a utilității acestuia și a regulilor de funcționare

Aplicația gestionează organizarea examenelor naționale (ex. Bacalaureat sau Evaluarea Națională) la nivelul unui județ. Sistemul ține evidența școlilor, a elevilor înscriși, a sălilor de examen și a profesorilor (supraveghetori și evaluatori).

Reguli de funcționare:

- Fiecare candidat provine de la o anumită unitate de învățământ.
- Școlile aparțin de un județ. Școlile mai mici pot fi subordonate unei școli principale.
- Un examen are mai multe probe (numite „sesiuni”, ex: proba de Română, proba de Matematică).
- Un profesor este repartizat să supravegheze într-o anumită sală, la o probă specifică.
- Fiecare probă susținută de un elev generează o lucrare, care este ulterior corectată de profesori evaluatori.
-

2. Prezentarea constrângerilor (restricții, reguli) impuse asupra modelului

- Unicitate: CNP-ul elevului trebuie să fie unic. Nu pot exista doi candidați cu același CNP.
- Obligatorietate: O lucrare trebuie legată obligatoriu de un candidat valid și de o probă specifică (NOT NULL).
- Validare: Nota acordată pe o lucrare trebuie să fie o valoare cuprinsă între 1.00 și 10.00 (CHECK).
-

3. Descrierea entităților, incluzând precizarea cheii primare

JUDET:

- județul în care se organizează examenele și care coordonează centrele și unitățile de învățământ.
- cheie primară: `id_judet`

CENTRU_EXAMEN:

- locatia coordonatoare sau zonala unde se pot desfasura probele de examen pentru candidatii din judet.
- cheie primara: `id_centru`

UNITATE_INVATAMANT:

- scoala sau liceul de care apartin candidatii. Poate fi o scoala cu personalitate juridica sau o structura arondata (subordonata).
- cheie primara: `id_unitate`

CANDIDAT:

- elevul inscris pentru a sustine examenele nationale.
- cheie primara: `id_candidat`

EXAMEN:

- tipul de examen general sustinut in anul respectiv (ex: Bacalaureat, Evaluare Nationala).
- cheie primara: `id_examen`

DISCIPLINA:

- materia specifica la care se sustine proba (ex: Limba Romana, Matematica).
- cheie primara: `id_disciplina`

SESIUNE:

- proba efectiva, programata la o anumita data, care asociaza un examen cu o anumita disciplina.
- cheie primara: `id_sesiune`

SALA:

- incaperea din cadrul unui centru de examen unde candidatii sustin proba si profesorii supravegheaza.
- cheie primara: `id_sala`

PROFESOR:

- cadrul didactic implicat in organizarea examenului (poate actiona ca supraveghetor sau ca evaluator).
- cheie primara: `id_profesor`

REPART_SUPRAV:

- entitate asociativa (rezultata dintr-o relatie ternara) care tine evidenta repartizarii profesorilor asistenti/supraveghetori in anumite sali, in timpul unei sesiuni specifice.
- cheie primara: id_repartizare (cheie artificiala)

LUCRARE:

- teza fizica (sau digitalizata) redactata de un candidat in cadrul unei sesiuni.
- cheie primara: id_lucrare

EVALUARE:

- entitate care retine nota acordata pe o lucrare specifica, de catre un anumit profesor evaluator.
- cheie primara: id_evaluare

4. Descrierea relațiilor, incluzând precizarea cardinalității acestora

RELATIE	CARDINALITATE	Observatii
contine	JUDET - UNITATE_INVATAMANT: one- to-many	Un judet are mai multe scoli. O scoala apartine de un singur judet.
subordine	UNITATE_INVATAMANT - UNITATE_INVATAMANT: one- to-many (Recursiva)	O scoala principala poate avea in subordine mai multe structuri (scoli arundate).
coordoneaza	JUDET - CENTRU_EXAMEN: one-to-many	Un judet are in subordine mai multe centre de examen zonale.
inscrie	UNITATE_INVATAMANT - CANDIDAT: one-to-many	O scoala are mai multi candidati inscrisi la examene.

RELATIE	CARDINALITATE	Observatii
dispune_de	CENTRU_EXAMEN - SALA: one-to-many	Un centru de examen are mai multe sali disponibile.
este_compus_din	EXAMEN - SESIUNE: one-to-many	Un examen (ex: Bacalaureat) contine mai multe sesiuni/probe (romana, mate etc).
se_sustine_in	DISCIPLINA - SESIUNE: one-to-many	O disciplina poate fi sustinuta in una sau mai multe sesiuni (ex: vara / toamna).
redacteaza	CANDIDAT - LUCRARE: one-to-many	Un candidat poate scrie mai multe lucrari (cate una pentru fiecare proba/sesiune).
genereaza	SESIUNE - LUCRARE: one-to-many	In cadrul unei sesiuni de examen se redacteaza multiple lucrari.
primeste	LUCRARE - EVALUARE: one-to-many	O lucrare poate fi evaluata (de mai multi profesori, rezultand mai multe note care se mediaza).
acorda_nota	PROFESOR - EVALUARE: one-to-many	Un profesor evaluator corecteaza si da note pentru mai multe lucrari.

RELATIE	CARDINALITATE	Observatii
repartizare_ternara	PROFESOR - SALA - SESIUNE: many-to-many-to-many	Mai multi profesori supravegheaza in mai multe sali la diverse sesiuni. Rezolvata prin entitatea REPART_SUPRAV.

•

5. Descrierea atributelor, incluzand tipul de date si eventualele constrangeri, valori implicite, valori posibile ale atributelor

JUDET

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_judet	NUMBER(10)	PK	1, 2, 3		generat prin secventa
nume_judet	VARCHAR2(50)	NOT NULL	Bucuresti , Cluj		denumire a judetului

CENTRU_EXAMEN

atribut	tip de date	constrangeri	valori posibile / exemple	valori implicit e	observatii
id_centru	NUMBER(10)	PK	6, 7		generat prin secventa
nume_centru	VARCHAR2(100)	NOT NULL	Centrul Zonal 1		
id_judet	NUMBER(10)	FK, NOT NULL	1, 2		leaga centrul de judetul sau

UNITATE_INVATAMANT

atribut	tip de date	constrangeri	valori posibile/ exempl e	valori implicit e	observatii
id_unitate	NUMBER(10)	PK	11, 12		
denumire	VARCHAR2(100)	NOT NULL	Col. Sf. Sava		numele scolii/liceul ui

atribut	tip de date	constrange ri	valori posibil e/ exempl e	valori implicit e	observatii
id_judet	NUMBER(10)	FK, NOT NULL	1, 2		
id_unitate_pari nte	NUMBER(10)	FK	11, NULL	NULL	NULL = e unitate principala

CANDIDAT

atribut	tip de date	constrang eri	valori posibile/ exemple	valori implici te	observatii
id_candidat	NUMBER(10)	PK	16, 17		
nume_comp let	VARCHAR2(1 00)	NOT NULL	Popescu Ion		
cnp	VARCHAR2(1 3)	UNIQUE, NOT NULL	50401011234 56		identificat or unic obligatori u

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicit e	observatii
id_unitate	NUMBER(10)	FK, NOT NULL	11, 12		liceul de provenien ta

EXAMEN

atribut	tip de date	constrange ri	valori posibile/ exemple	valori implicit e	observati i
id_examen	NUMBER(10)	PK	21, 22		
denumire_exam en	VARCHAR2(5 0)	NOT NULL	Bacalaure at		
an_sustinere	NUMBER(4)	NOT NULL	2026		anul organizar ii

DISCIPLINA

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_disciplina	NUMBER(10)	PK	26, 27		
denumire_disciplina	VARCHAR2(50)	NOT NULL	Matematica, Istorie		

SESIUNE

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_sesiune	NUMBER(10)	PK	31, 32		
data_examen	DATE	NOT NULL	15-JUN-2026		data exacta a probei
id_examen	NUMBER(10)	FK, NOT NULL	21		
id_disciplina	NUMBER(10)	FK, NOT NULL	26		

SALA

atribut	tip de date	constrange ri	valori posibile/ exemple	valori implicit e	observatii
id_sala	NUMBER(10)	PK	36, 37		
denumire_sala	VARCHAR2(50)	NOT NULL	Sala 101, Amfiteatru		
capacitate	NUMBER(3)		30, 50		locuri disponibil e
id_centru	NUMBER(10)	FK, NOT NULL	6, 7		

PROFESOR

atribut	tip de date	constranger i	valori posibile/ exemple	valori implicit e	observati i
id_profesor	NUMBER(10)	PK	41, 42		
nume_profesor	VARCHAR2(100)	NOT NULL	Avram Vasile		

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicit e	observatii
grad_didactic	VARCHAR2(20)		Grad I, Debutan t		

REPART_SUPRAV

atribut	tip de date	constrangeri	valori posibile / exempl e	valori implicit e	observatii
id_repartizar e	NUMBER(10)	PK	46, 47		cheie primara artificiala
id_profesor	NUMBER(10)	FK, NOT NULL	41		profesorul supravegheto r
id_sala	NUMBER(10)	FK, NOT NULL	36		sala alocata
id_sesiune	NUMBER(10)	FK, NOT NULL	31		proba la care participa

LUCRARE

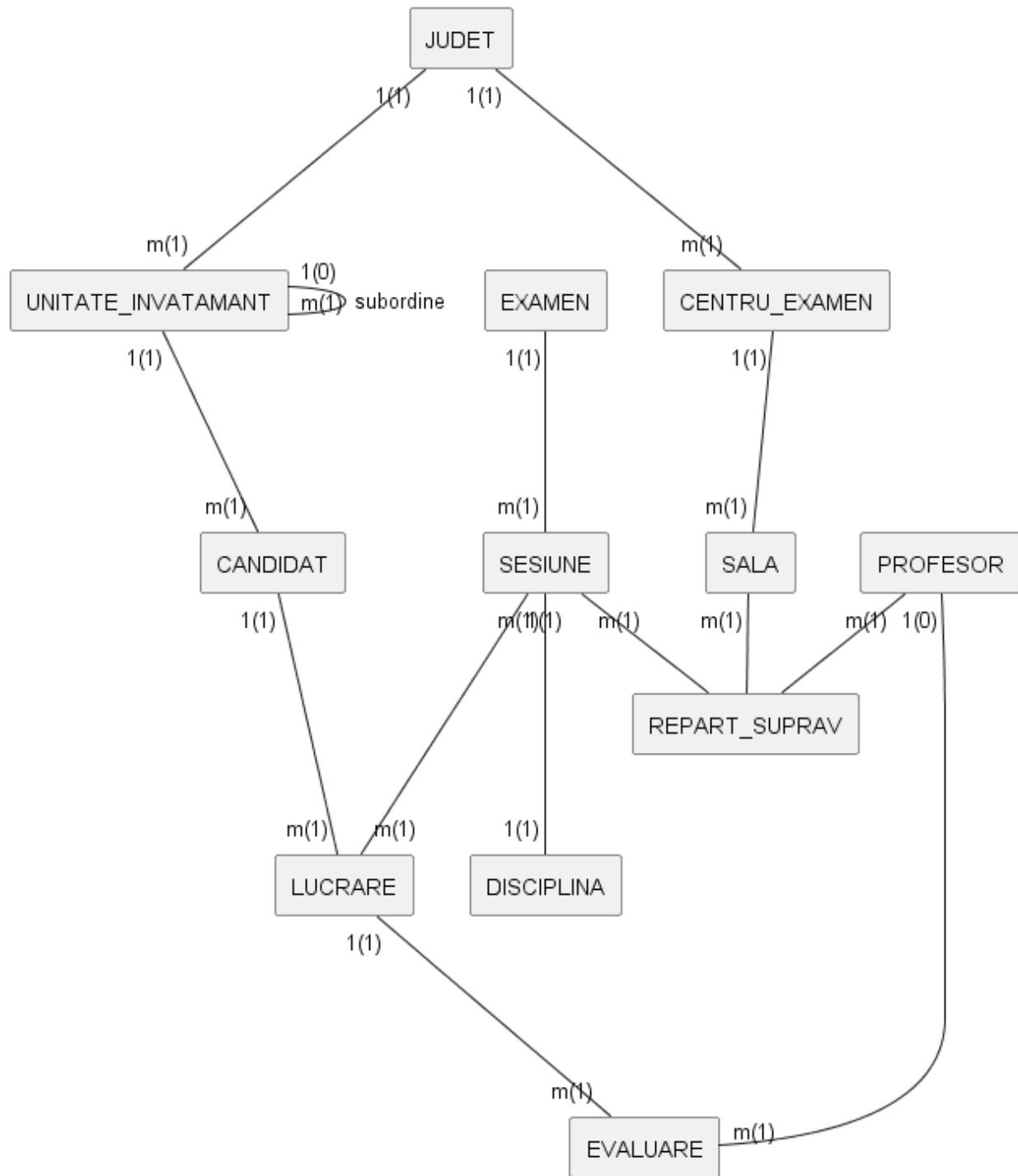
atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_lucrare	NUMBER(10)	PK	56, 57		
cod_secret	VARCHAR2(20)		SEC123, SEC124		colt negru / cod bare
id_candidat	NUMBER(10)	FK, NOT NULL	16		
id_sesiune	NUMBER(10)	FK, NOT NULL	31		

EVALUARE

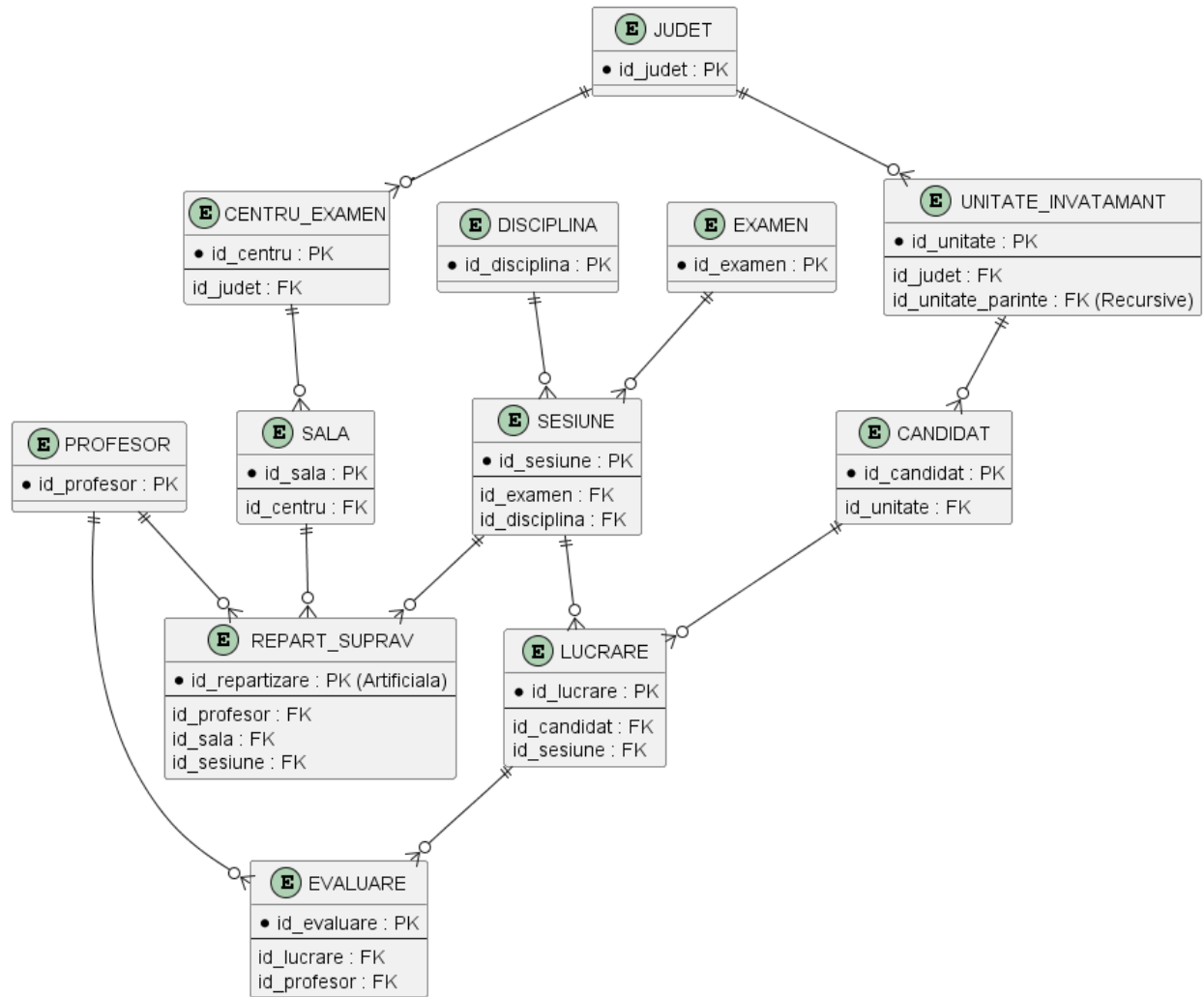
atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_evaluare	NUMBER(10)	PK	61, 62		
nota_acordata	NUMBER(4,2)	CHECK(1- 10)	9.50, 10.00		nota data de evaluator

atribut	tip de date	constrangeri	valori posibile/ exemple	valori implicite	observatii
id_lucrare	NUMBER(10)	FK, NOT NULL	56		
id_profesor	NUMBER(10)	FK, NOT NULL	41		

6. Diagrama Entitate-Relație (ERD)



7. Diagrama Conceptuală



8. Enumerarea schemelor relationale

- JUDET(#id_judet, nume_judet)
- CENTRU_EXAMEN(#id_centru, nume_centru, id_judet)
- UNITATE_INVATAMANT(#id_unitate, denumire, id_judet, id_unitate_parinte)
- CANDIDAT(#id_candidat, nume_complet, cnp, id_unitate)
- EXAMEN(#id_examen, denumire_examen, an_sustinere)
- DISCIPLINA(#id_disciplina, denumire_disciplina)
- SESIUNE(#id_sesiune, data_examen, id_examen, id_disciplina)
- SALA(#id_sala, denumire_sala, capacitate, id_centru)
- PROFESOR(#id_profesor, nume_profesor, grad_didactic)
- REPART_SUPRAV(#id_repartizare, id_profesor, id_sala, id_sesiune)
- LUCRARE(#id_lucrare, cod_secret, id_candidat, id_sesiune)
- EVALUARE(#id_evaluare, nota_acordata, id_lucrare, id_profesor)

9. Normalizarea (FN1 - FN3)

Aducerea în FN1, FN2, FN3 prin descompuneri succesive fără pierdere de informație, pornind de la o structură nenormalizată ipotetică, pentru a ajunge la schemele din diagrama conceptuală.

Situația inițială (Relație nenormalizată): SITUATIE_EXAMEN(id_lucrare, id_profesor, nota_acordata, id_candidat, nume_complet, id_unitate, denumire, limbi_straine_cunoscute)

A. Trecerea în FN1 Atributul limbi_straine_cunoscute este un atribut multiplu (ex: "Engleză, Franceză"), deci relația SITUATIE_EXAMEN nu este în FN1. Pentru a o aduce în FN1, separăm atributul multiplu:

- LIMBI_CANDIDAT(id_candidat, limba_straina)
- SITUATIE_EVALUARE(id_lucrare, id_profesor, nota_acordata, id_candidat, nume_complet, id_unitate, denumire) - este în FN1.

B. Trecerea în FN2 Analizăm dependențele funcționale din tabelul SITUATIE_EVALUARE, care are cheia primară compusă (id_lucrare, id_profesor): $F = \{ (id_lucrare, id_profesor) \rightarrow nota_acordata, id_lucrare \rightarrow (id_candidat, nume_complet, id_unitate, denumire), id_candidat \rightarrow (nume_complet, id_unitate, denumire), id_unitate \rightarrow denumire \}$

Relația nu este în FN2. Avem o dependență parțială: attributele id_candidat, nume_complet, id_unitate și denumire depind doar de id_lucrare, care este doar o parte a cheii candidate. Descompunem relația în:

- EVALUARE(id_lucrare, id_profesor, nota_acordata) - este în FN3.
- LUCRARE_CANDIDAT(id_lucrare, id_candidat, nume_complet, id_unitate, denumire) - este în FN2.

C. Trecerea în FN3 Relația LUCRARE_CANDIDAT nu este în FN3, deoarece conține dependențe tranzitive (attribute care depind de alte attribute non-cheie):

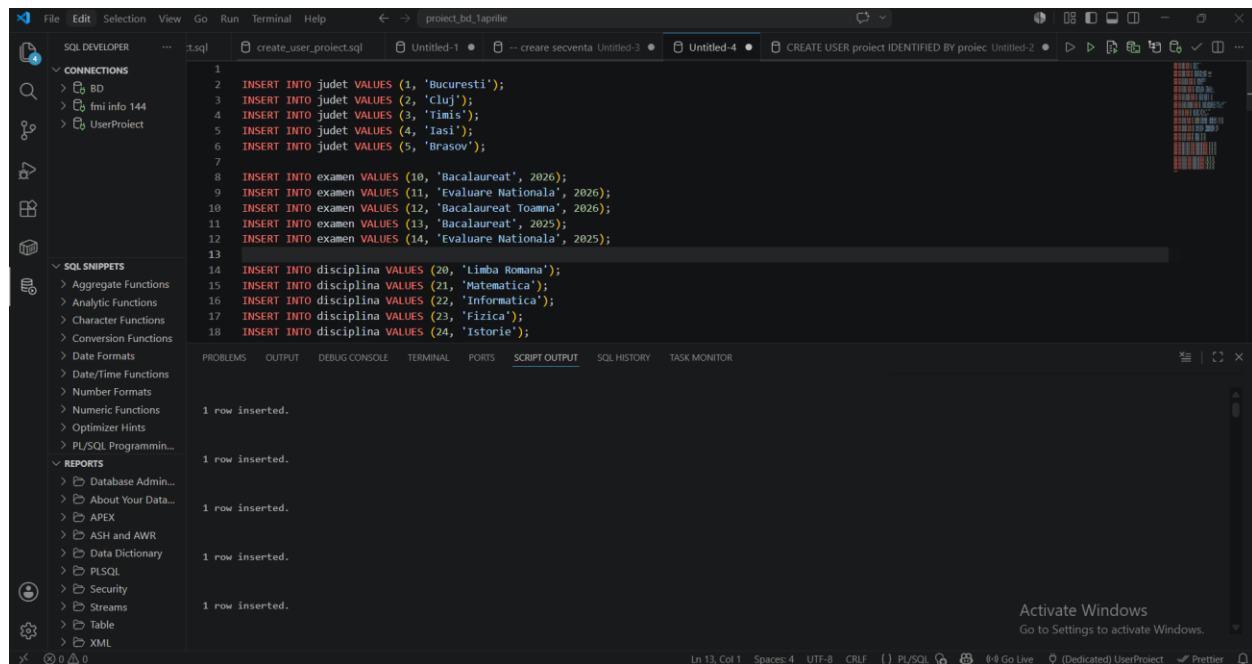
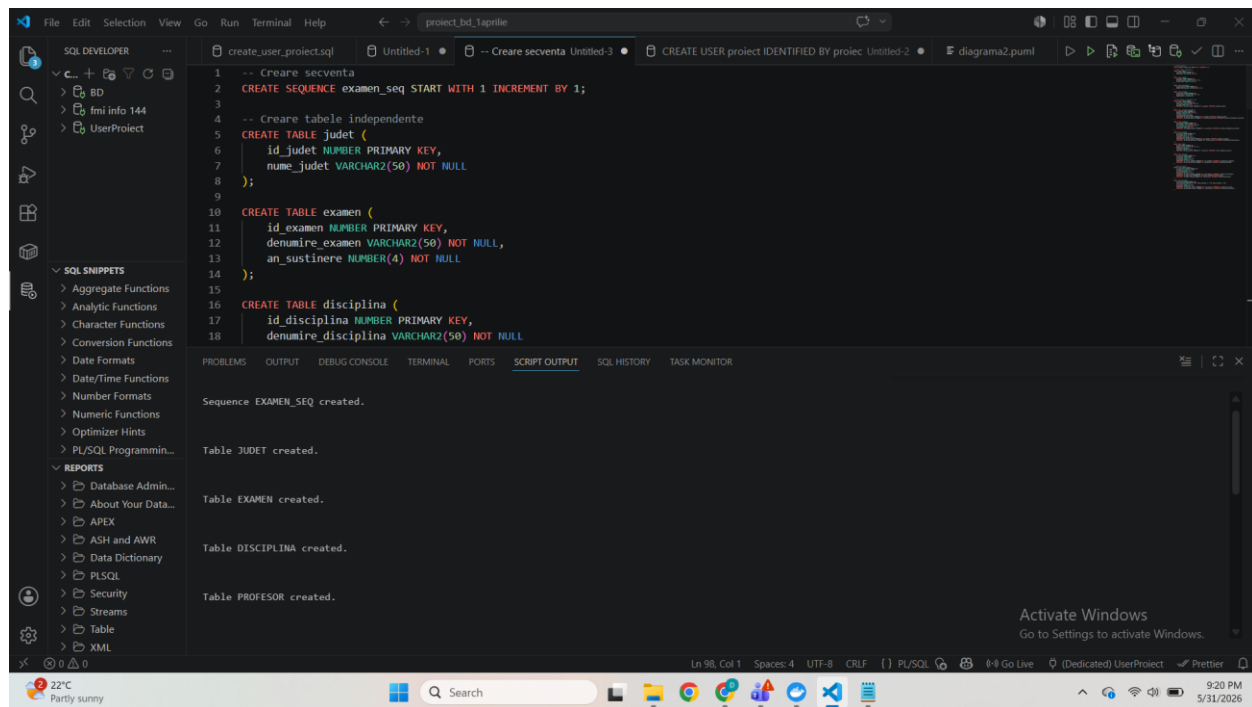
- id_lucrare $\rightarrow$ id_candidat $\rightarrow$ nume_complet
- id_candidat $\rightarrow$ id_unitate $\rightarrow$ denumire

Descompunem relația pentru a elimina dependențele tranzitive, obținând exact tabelele noastre finale:

- LUCRARE(id_lucrare, id_candidat) - este în FN3.
- CANDIDAT(id_candidat, nume_complet, id_unitate) - este în FN3.
- UNITATE_INVATAMANT(id_unitate, denumire) - este în FN3.

10. Crearea unei secvențe ce va fi utilizată în inserarea înregistrărilor în tabele (punctul 11).

11. Crearea tabelelor în SQL și inserarea de **date coerente** în fiecare dintre acestea (minimum 5 înregistrări în fiecare tabel neasociativ; minimum 10 înregistrări în tabelele asociative; maxim 30 de înregistrări în fiecare tabel).



-- creare secventa

CREATE SEQUENCE examen_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE judet (  
    id_judet NUMBER PRIMARY KEY,  
    nume_judet VARCHAR2(50) NOT NULL  
);
```

```
CREATE TABLE examen (  
    id_examen NUMBER PRIMARY KEY,  
    denumire_examen VARCHAR2(50) NOT NULL,  
    an_sustinere NUMBER(4) NOT NULL  
);
```

```
CREATE TABLE disciplina (  
    id_disciplina NUMBER PRIMARY KEY,  
    denumire_disciplina VARCHAR2(50) NOT NULL  
);
```

```
CREATE TABLE profesor (  
    id_profesor NUMBER PRIMARY KEY,  
    nume_profesor VARCHAR2(100) NOT NULL,  
    grad_didactic VARCHAR2(20)  
);
```

```
-- creare tabele cu dependente (FK)
```

```

CREATE TABLE centru_examen (

    id_centru NUMBER PRIMARY KEY,

    nume_centru VARCHAR2(100) NOT NULL,

    id_judet NUMBER NOT NULL,

    CONSTRAINT fk_centru_judet FOREIGN KEY (id_judet) REFERENCES judet(id_judet)

);

```

```

CREATE TABLE unitate_invatamant (

    id_unitate NUMBER PRIMARY KEY,

    denumire VARCHAR2(100) NOT NULL,

    id_judet NUMBER NOT NULL,

    id_unitate_parinte NUMBER,

    CONSTRAINT fk_unitate_judet FOREIGN KEY (id_judet) REFERENCES judet(id_judet),

    CONSTRAINT fk_unitate_parinte FOREIGN KEY (id_unitate_parinte) REFERENCES unitate_invatamant(id_unitate)

);

```

```

CREATE TABLE candidat (

    id_candidat NUMBER PRIMARY KEY,

    nume_complet VARCHAR2(100) NOT NULL,

    cnp VARCHAR2(13) UNIQUE NOT NULL,

    id_unitate NUMBER NOT NULL,

    CONSTRAINT fk_candidat_unitate FOREIGN KEY (id_unitate) REFERENCES unitate_invatamant(id_unitate)

);

```

```

CREATE TABLE sesiune (

```

```
id_sesiune NUMBER PRIMARY KEY,  
  
data_examen DATE NOT NULL,  
  
id_examen NUMBER NOT NULL,  
  
id_disciplina NUMBER NOT NULL,  
  
CONSTRAINT fk_sesiune_examen FOREIGN KEY (id_examen) REFERENCES examen(id_examen),  
  
CONSTRAINT fk_sesiune_disciplina FOREIGN KEY (id_disciplina) REFERENCES disciplina(id_disciplina)  
  
);
```

```
CREATE TABLE sala (  
  
id_sala NUMBER PRIMARY KEY,  
  
denumire_sala VARCHAR2(50) NOT NULL,  
  
capacitate NUMBER(3),  
  
id_centru NUMBER NOT NULL,  
  
CONSTRAINT fk_sala_centru FOREIGN KEY (id_centru) REFERENCES centru_examen(id_centru)  
  
);
```

```
CREATE TABLE lucrare (  
  
id_lucrare NUMBER PRIMARY KEY,  
  
cod_secret VARCHAR2(20),  
  
id_candidat NUMBER NOT NULL,  
  
id_sesiune NUMBER NOT NULL,  
  
CONSTRAINT fk_lucrare_candidat FOREIGN KEY (id_candidat) REFERENCES candidat(id_candidat),  
  
CONSTRAINT fk_lucrare_sesiune FOREIGN KEY (id_sesiune) REFERENCES sesiune(id_sesiune)  
  
);
```

-- tabele asociative

```
CREATE TABLE repart_suprav (  
    id_repartizare NUMBER PRIMARY KEY,  
    id_profesor NUMBER NOT NULL,  
    id_sala NUMBER NOT NULL,  
    id_sesiune NUMBER NOT NULL,  
    CONSTRAINT fk_repart_profesor FOREIGN KEY (id_profesor) REFERENCES profesor(id_profesor),  
    CONSTRAINT fk_repart_sala FOREIGN KEY (id_sala) REFERENCES sala(id_sala),  
    CONSTRAINT fk_repart_sesiune FOREIGN KEY (id_sesiune) REFERENCES sesiune(id_sesiune)  
);  
  
CREATE TABLE evaluare (  
    id_evaluare NUMBER PRIMARY KEY,  
    nota_acordata NUMBER(4,2) CHECK (nota_acordata >= 1 AND nota_acordata <= 10),  
    id_lucrare NUMBER NOT NULL,  
    id_profesor NUMBER NOT NULL,  
    CONSTRAINT fk_evaluare_lucrare FOREIGN KEY (id_lucrare) REFERENCES lucrare(id_lucrare),  
    CONSTRAINT fk_evaluare_profesor FOREIGN KEY (id_profesor) REFERENCES profesor(id_profesor)  
);
```

```
INSERT INTO judet VALUES (1, 'Bucuresti');
```

```
INSERT INTO judet VALUES (2, 'Cluj');
```

```
INSERT INTO judet VALUES (3, 'Timis');
```

INSERT INTO judet VALUES (4, 'Iasi');

INSERT INTO judet VALUES (5, 'Brasov');

INSERT INTO examen VALUES (10, 'Bacalaureat', 2026);

INSERT INTO examen VALUES (11, 'Evaluare Nationala', 2026);

INSERT INTO examen VALUES (12, 'Bacalaureat Toamna', 2026);

INSERT INTO examen VALUES (13, 'Bacalaureat', 2025);

INSERT INTO examen VALUES (14, 'Evaluare Nationala', 2025);

INSERT INTO disciplina VALUES (20, 'Limba Romana');

INSERT INTO disciplina VALUES (21, 'Matematica');

INSERT INTO disciplina VALUES (22, 'Informatica');

INSERT INTO disciplina VALUES (23, 'Fizica');

INSERT INTO disciplina VALUES (24, 'Istorie');

INSERT INTO profesor VALUES (30, 'Popescu Ion', 'Grad I');

INSERT INTO profesor VALUES (31, 'Ionescu Maria', 'Grad II');

INSERT INTO profesor VALUES (32, 'Marin Vasile', 'Debutant');

INSERT INTO profesor VALUES (33, 'Dumitru Elena', 'Grad I');

INSERT INTO profesor VALUES (34, 'Stan Andrei', 'Definitivat');

INSERT INTO centru_examen VALUES (40, 'Centrul Zonal 1', 1);

INSERT INTO centru_examen VALUES (41, 'Centrul Zonal 2', 1);

INSERT INTO centru_examen VALUES (42, 'Centrul Zonal 3', 2);

INSERT INTO centru_examen VALUES (43, 'Centrul Zonal 4', 3);

INSERT INTO centru_examen VALUES (44, 'Centrul Zonal 5', 4);

INSERT INTO unitate_invatamant VALUES (50, 'Colegiul National Sf. Sava', 1, NULL);

INSERT INTO unitate_invatamant VALUES (51, 'Scoala Gimnaziala Nr. 49', 1, 50);

INSERT INTO unitate_invatamant VALUES (52, 'Liceul Emil Racovita', 2, NULL);

INSERT INTO unitate_invatamant VALUES (53, 'Colegiul Banatean', 3, NULL);

INSERT INTO unitate_invatamant VALUES (54, 'Liceul National Iasi', 4, NULL);

INSERT INTO sala VALUES (60, 'Amfiteatrul Mare', 50, 40);

INSERT INTO sala VALUES (61, 'Sala 101', 30, 40);

INSERT INTO sala VALUES (62, 'Sala 102', 30, 41);

INSERT INTO sala VALUES (63, 'Laborator Info', 25, 42);

INSERT INTO sala VALUES (64, 'Sala Sport', 100, 43);

INSERT INTO sesiune VALUES (70, TO_DATE('15-06-2026', 'DD-MM-YYYY'), 10, 20);

INSERT INTO sesiune VALUES (71, TO_DATE('17-06-2026', 'DD-MM-YYYY'), 10, 21);

INSERT INTO sesiune VALUES (72, TO_DATE('19-06-2026', 'DD-MM-YYYY'), 10, 22);

INSERT INTO sesiune VALUES (73, TO_DATE('22-06-2026', 'DD-MM-YYYY'), 11, 20);

INSERT INTO sesiune VALUES (74, TO_DATE('24-06-2026', 'DD-MM-YYYY'), 11, 21);

INSERT INTO candidat VALUES (80, 'Avram Mihai', '5040101123456', 50);

INSERT INTO candidat VALUES (81, 'Badea Alina', '6040202123456', 50);

INSERT INTO candidat VALUES (82, 'Cretu George', '5050303123456', 52);

INSERT INTO candidat VALUES (83, 'Dima Florin', '5040404123456', 53);

INSERT INTO candidat VALUES (84, 'Ene Larisa', '6050505123456', 54);

INSERT INTO lucrare VALUES (90, 'SEC111', 80, 70);

INSERT INTO lucrare VALUES (91, 'SEC222', 80, 71);

INSERT INTO lucrare VALUES (92, 'SEC333', 81, 70);

INSERT INTO lucrare VALUES (93, 'SEC444', 82, 70);

INSERT INTO lucrare VALUES (94, 'SEC555', 83, 73);

INSERT INTO lucrare VALUES (95, 'SEC666', 84, 74);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 30, 60, 70);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 31, 60, 70);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 32, 61, 70);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 33, 62, 71);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 34, 63, 72);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 30, 64, 73);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 31, 64, 73);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 32, 60, 74);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 33, 61, 74);

INSERT INTO repart_suprav VALUES (examen_seq.NEXTVAL, 34, 62, 74);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 9.50, 90, 30);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 9.00, 90, 31);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 8.50, 91, 32);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 8.00, 91, 33);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 10.00, 92, 34);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 9.80, 92, 30);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 7.50, 93, 31);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 7.50, 93, 32);

INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 6.00, 94, 33);

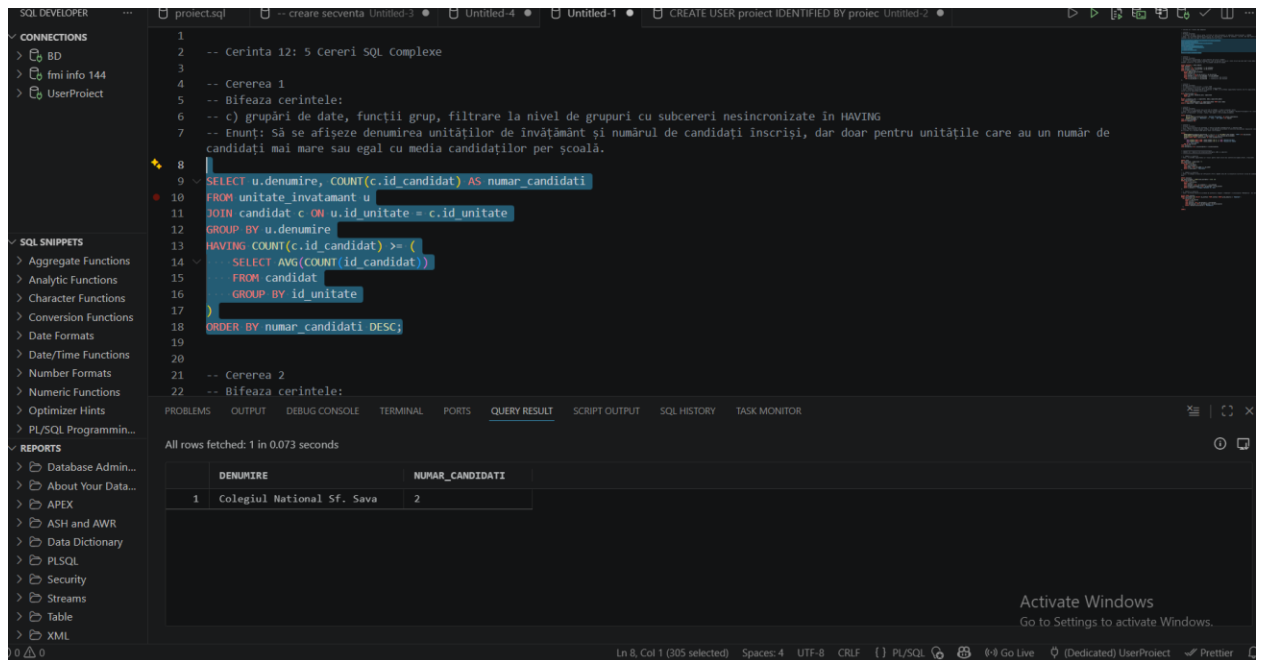
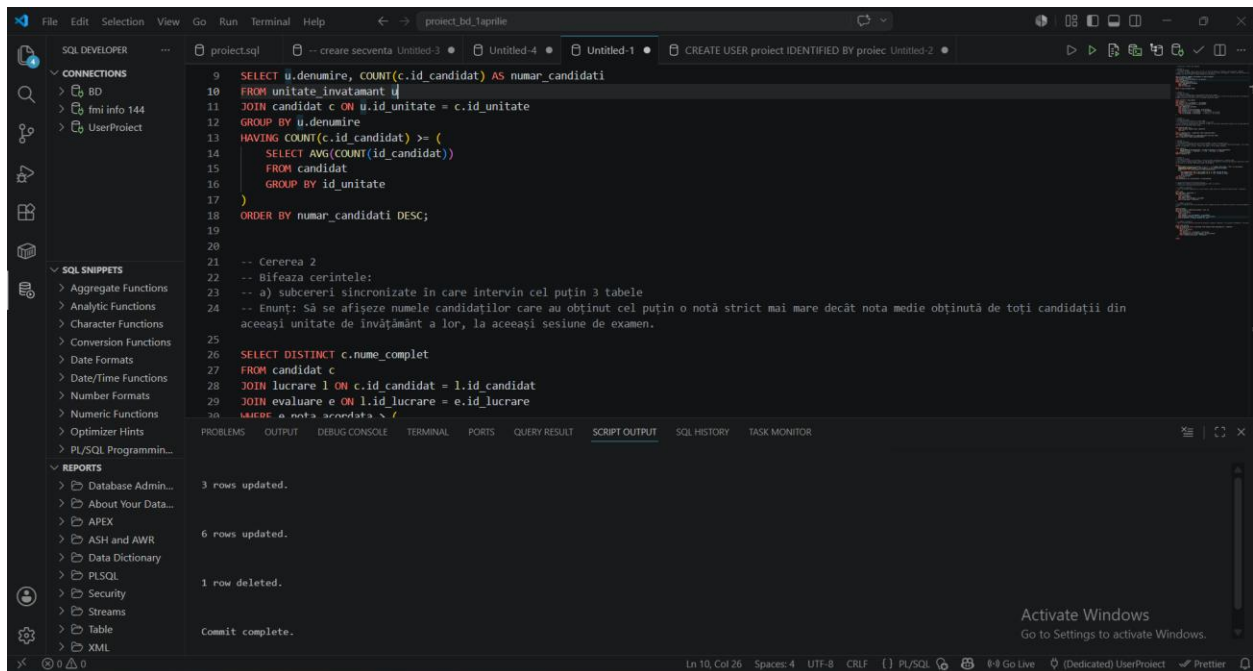
INSERT INTO evaluare VALUES (examen_seq.NEXTVAL, 8.20, 95, 34);

COMMIT;

12. Formulați în limbaj natural și implementați 5 cereri SQL complexe ce vor utiliza, în ansamblul lor, următoarele elemente:

- a. subcereri sincronizate în care intervin cel puțin 3 tabele
- b. subcereri nesincronizate în clauza FROM
- c. grupări de date, funcții grup, filtrare la nivel de grupuri cu subcereri nesincronizate (în clauza de HAVING)
- d. ordonări și utilizarea funcțiilor NVL și DECODE (în cadrul aceleiași cereri)
- e. utilizarea a cel puțin 2 funcții pe șiruri de caractere, 2 funcții pe date calendaristice, a cel puțin unei expresii CASE
- f. utilizarea a cel puțin 1 bloc de cerere (clauza WITH)

13. Implementarea a 3 operații de actualizare și de suprimare a datelor utilizând subcereri.



23 -- a) subcereri sincronizate in care intervin cel puțin 3 tabele
24 -- Enunț: Să se afișeze numele candidaților care au obținut cel puțin o notă strict mai mare decât nota medie obținută de toți candidații din
25 -- aceeași unitate de învățământ a lor, la aceeași sesiune de examen.

```
26 SELECT DISTINCT c.nume_complet
27 FROM candidat c
28 JOIN lucrare l ON c.id_candidat = l.id_candidat
29 JOIN evaluare e ON l.id_lucrare = e.id_lucrare
30 WHERE e.nota_acordata > (
31     SELECT AVG(e2.nota_acordata)
32     FROM evaluare e2
33     JOIN lucrare l2 ON e2.id_lucrare = l2.id_lucrare
34     JOIN candidat c2 ON l2.id_candidat = c2.id_candidat
35     WHERE c2.id_unitate = c.id_unitate -- corelație 1 (pe unitate)
36     AND l2.id_sesiune = l.id_sesiune -- corelație 2 (pe sesiune)
37 )
38
39
40 -- Cererea 3
41 -- Bifeaza cerintele:
42 -- b) subcereri nesincronizate in clauza FROM
43 -- f) utilizarea a cel puțin 1 bloc de cerere (clauza WITH)
44 -- Enunț: Folosind clauza WITH și o subcerere in FROM, să se afișeze capacitatea fiecărei săli și capacitatea medie a tuturor sălilor, pentru
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 2 in 0.114 seconds

	NUME_COMPLET
1	Auram Mihai
2	Badea Alina

40 -- Cererea 3
41 -- Bifeaza cerintele:
42 -- b) subcereri nesincronizate in clauza FROM
43 -- f) utilizarea a cel puțin 1 bloc de cerere (clauza WITH)
44 -- Enunț: Folosind clauza WITH și o subcerere in FROM, să se afișeze capacitatea fiecărei săli și capacitatea medie a tuturor sălilor, pentru
45 -- sălile care au capacitatea peste medie.

```
46 WITH StatisticSali AS (
47     SELECT id_sala, denumire_sala, capacitate
48     FROM sala
49 )
50 SELECT s.denumire_sala, s.capacitate, medii.capacitate_medie
51 FROM StatisticSali s,
52 (SELECT AVG(capacitate) AS capacitate_medie FROM sala) medii
53 WHERE s.capacitate > medii.capacitate_medie;
54
55
56 -- Cererea 4
57 -- Bifeaza cerintele:
58 -- d) ordonari și utilizarea funcțiilor NVL și DECODE in cadrul aceleiași cereri
59 -- Enunț: Să se afișeze școlile, ID-ul școlii părinte (dacă nu au, se afișează 'Unitate Principala') și o etichetă (DECODE) pentru județul de care
-- aparțin (1='Bucuresti', 2='Cluj', restul 'Alt judet'). Se va ordona alfabetic.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 2 in 0.061 seconds

	DENUMIRE_SALA	CAPACITATE	CAPACITATE_MEDIE
1	Amfiteatrul Mare	60	53
2	Sala Sport	100	53

Activate Windows
Go to Settings to activate Windows.

```

46 WITH Statisticisali AS (
47     SELECT id_sala, denumire_sala, capacitate
48     FROM sala
49 )
50 SELECT s.denumire_sala, s.capacitate, medii.capacitate_medie
51 FROM Statisticisali s,
52      (SELECT AVG(capacitate) AS capacitate_medie FROM sala) medii
53 WHERE s.capacitate > medii.capacitate_medie;
54
55 -- Cererea 4
56 -- Bifeaza cerintele:
57 -- d) ordonari și utilizarea funcțiilor NVL și DECODE în cadrul aceleiași cereri
58 -- Enunț: Să se afișeze școlile, ID-ul școlii părinte (dacă nu au, se afișează 'Unitate Principala') și o etichetă (DECODE) pentru județul de care
59 -- aparțin (1='Bucuresti', 2='Cluj', restul 'Alt judet'). Se va ordona alfabetic.
60
61 SELECT denumire,
62        NVL(TO_CHAR(id_unitate_părinte), 'Unitate Principala') AS status_subordonare,
63        DECODE(id_judet, 1, 'Bucuresti', 2, 'Cluj', 'Alt judet') AS regiune
64 FROM unitate_invatamant
65 ORDER BY denumire ASC;
66
67

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 5 in 0.058 seconds

	DENUMIRE	STATUS_SUBORDONARE	REGIUNE
1	Colegiul Banatean	Unitate Principala	Alt judet
2	Colegiul National Sf. Sava	Unitate Principala	Bucuresti
3	Liceul Emil Racovita	Unitate Principala	Cluj
4	Liceul National Iasi	Unitate Principala	Alt judet
5	Scoala Gimnaziala Nr. 49	50	Bucuresti

Activate Windows
Go to Settings to activate Windows.

Ln 60, Col 1 (231 selected) Spaces: 4 UTF-8 CRLF {} PL/SQL 8-9 Go Live (Dedicated) UserProject Prettier

```

65 ORDER BY denumire ASC;
66
67 -- Cererea 5
68 -- Bifeaza cerintele:
69 -- e) 2 funcții pe șiruri de caractere, 2 funcții pe date calendaristice, o expresie CASE
70 -- Enunț: Pentru fiecare sesiune, afișați un cod generat din primele 3 litere ale disciplinei (majuscule) concatenate cu anul, câte luni au trecut
71 -- de la probă, și o etichetă CASE pentru sezon (vară/toamnă).
72
73 SELECT
74     UPPER(SUBSTR(d.denumire_disciplina, 1, 3)) || '-' || TO_CHAR(s.data_examen, 'YYYY') AS cod_sesiune,
75     ROUND(MONTHS_BETWEEN(SYSDATE, s.data_examen)) AS luni_trecute_de_la_examen,
76     EXTRACT(MONTH FROM s.data_examen) AS luna_calendaristica,
77     CASE
78         WHEN EXTRACT(MONTH FROM s.data_examen) IN (6, 7) THEN 'Sesiune de Vara'
79         WHEN EXTRACT(MONTH FROM s.data_examen) IN (8, 9) THEN 'Sesiune de Toamna'
80         ELSE 'Alte sesiuni'
81     END AS tip_sesiune
82 FROM sesiune s
83 JOIN disciplina d ON s.id_disciplina = d.id_disciplina;
84
85

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 5 in 0.076 seconds

	COD_SESIUNE	LUNI_TRECUTE_DE_LA_EXAMEN	LUNA_CALENDARISTICA	TIP_SESIUNE
1	LIM-2026	0	6	Sesiune de Vara
2	LIM-2026	-1	6	Sesiune de Vara
3	MAT-2026	-1	6	Sesiune de Vara
4	MAT-2026	-1	6	Sesiune de Vara
5	INF-2026	-1	6	Sesiune de Vara

Activate Windows
Go to Settings to activate Windows.

-- Cerinta 12: 5 Cereri SQL Complex

-- Cererea 1

-- Bifeaza cerintele:

-- c) grupări de date, funcții grup, filtrare la nivel de grupuri cu subcereri nesincronizate în HAVING

-- Enunț: Să se afișeze denumirea unităților de învățământ și numărul de candidați înscriși, dar doar pentru unitățile care au un număr de candidați mai mare sau egal cu media candidaților per școală.

```
SELECT u.denumire, COUNT(c.id_candidat) AS numar_candidati
FROM unitate_invatamant u
JOIN candidat c ON u.id_unitate = c.id_unitate
GROUP BY u.denumire
HAVING COUNT(c.id_candidat) >= (
    SELECT AVG(COUNT(id_candidat))
    FROM candidat
    GROUP BY id_unitate
)
ORDER BY numar_candidati DESC;
```

-- Cererea 2

-- Bifeaza cerintele:

-- a) subcereri sincronizate în care intervin cel puțin 3 tabele

-- Enunț: Să se afișeze numele candidaților care au obținut cel puțin o notă strict mai mare decât nota medie obținută de toți candidații din aceeași unitate de învățământ a lor, la aceeași sesiune de examen.

```
SELECT DISTINCT c.ume_complet
FROM candidat c
JOIN lucrare l ON c.id_candidat = l.id_candidat
JOIN evaluare e ON l.id_lucrare = e.id_lucrare
WHERE e.nota_acordata > (
    SELECT AVG(e2.nota_acordata)
```

```

FROM evaluare e2

JOIN lucrare l2 ON e2.id_lucrare = l2.id_lucrare

JOIN candidat c2 ON l2.id_candidat = c2.id_candidat

WHERE c2.id_unitate = c.id_unitate -- Corelație 1 (pe unitate)

      AND l2.id_sesiune = l.id_sesiune -- Corelație 2 (pe sesiune)

);

-- Cererea 3

-- Bifeaza cerintele:

-- b) subcereri nesincronizate în clauza FROM

-- f) utilizarea a cel puțin 1 bloc de cerere (clauza WITH)

-- Enunț: Folosind clauza WITH și o subcerere în FROM, să se afișeze capacitatea fiecărei săli și capacitatea
medie a tuturor sălilor, pentru sălile care au capacitatea peste medie.

WITH StatisticiSali AS (

    SELECT id_sala, denumire_sala, capacitate

    FROM sala

)

SELECT s.denumire_sala, s.capacitate, medii.capacitate_medie

FROM StatisticiSali s,

      (SELECT AVG(capacitate) AS capacitate_medie FROM sala) medii

WHERE s.capacitate > medii.capacitate_medie;

-- Cererea 4

```

-- Bifeaza cerintele:

-- d) ordonari și utilizarea funcțiilor NVL și DECODE în cadrul aceleiași cereri

-- Enunț: Să se afișeze școlile, ID-ul școlii părinte (dacă nu au, se afișează 'Unitate Principala') și o etichetă (DECODE) pentru județul de care aparțin (1='Bucuresti', 2='Cluj', restul 'Alt judet'). Se va ordona alfabetic.

SELECT denumire,

NVL(TO_CHAR(id_unitate_parinte), 'Unitate Principala') AS status_subordonare,

DECODE(id_judet, 1, 'Bucuresti', 2, 'Cluj', 'Alt judet') AS regiune

FROM unitate_invatamant

ORDER BY denumire ASC;

-- Cererea 5

-- Bifeaza cerintele:

-- e) 2 funcții pe șiruri de caractere, 2 funcții pe date calendaristice, o expresie CASE

-- Enunț: Pentru fiecare sesiune, afișați un cod generat din primele 3 litere ale disciplinei (majuscule) concatenate cu anul, câte luni au trecut de la probă, și o etichetă CASE pentru sezon (vară/toamnă).

SELECT

UPPER(SUBSTR(d.denumire_disciplina, 1, 3)) || '-' || TO_CHAR(s.data_examen, 'YYYY') AS
cod_sesiune,

ROUND(MONTHS_BETWEEN(SYSDATE, s.data_examen)) AS luni_trecute_de_la_examen,

EXTRACT(MONTH FROM s.data_examen) AS luna_calendaristica,

CASE

WHEN EXTRACT(MONTH FROM s.data_examen) IN (6, 7) THEN 'Sesiune de Vara'

WHEN EXTRACT(MONTH FROM s.data_examen) IN (8, 9) THEN 'Sesiune de Toamna'

ELSE 'Alte sesiuni'

```
END AS tip_sesiune  
  
FROM sesiune s  
  
JOIN disciplina d ON s.id_disciplina = d.id_disciplina;
```

-- Cerinta 13: 3 Operații de actualizare/stergere cu subcereri

-- 1. UPDATE cu subcerere

-- Enunt: Se mărește capacitatea cu 5 locuri pentru toate sălile din centrele de examen aflate în București.

```
UPDATE sala  
  
SET capacitate = capacitate + 5  
  
WHERE id_centru IN (  
  
    SELECT c.id_centru  
  
    FROM centru_examen c  
  
    JOIN judet j ON c.id_judet = j.id_judet  
  
    WHERE j.nume_judet = 'Bucuresti'  
  
);
```

-- 2. UPDATE cu subcerere

-- Enunt: Se adaugă un bonus de 0.50 puncte (fără a depăși nota 10) la evaluările lucrărilor scrise de candidații de la 'Colegiul National Sf. Sava'.

```
UPDATE evaluare  
  
SET nota_acordata = LEAST(nota_acordata + 0.50, 10)
```

```
WHERE id_lucrare IN (  
  
    SELECT l.id_lucrare  
  
    FROM lucrare l  
  
    JOIN candidat c ON l.id_candidat = c.id_candidat  
  
    JOIN unitate_invatamant u ON c.id_unitate = u.id_unitate  
  
    WHERE u.denumire = 'Colegiul National Sf. Sava'  
  
);
```

-- 3. DELETE cu subcerere

-- Enunt: Se sterg evaluările acordate de profesorii stagiaari ('Debutant') la disciplina 'Matematica', din motive de reevaluare.

```
DELETE FROM evaluare  
  
WHERE id_profesor IN (SELECT id_profesor FROM profesor WHERE grad_didactic = 'Debutant')  
  
AND id_lucrare IN (  
  
    SELECT l.id_lucrare  
  
    FROM lucrare l  
  
    JOIN sesiune s ON l.id_sesiune = s.id_sesiune  
  
    JOIN disciplina d ON s.id_disciplina = d.id_disciplina  
  
    WHERE d.denumire_disciplina = 'Matematica'  
  
);
```

```
COMMIT;
```

14. Crearea unei vizualizări complexe. Dați un exemplu de operație LMD permisă pe vizualizarea respectivă și un exemplu de operație LMD nepermisă.

-- Cerinta 14: Vizualizare complexa si operatii LMD

-- 1. Crearea vizualizării complexe (conține un JOIN)

-- Enunt: Sa se creeze o vizualizare care expune detaliile candidaților impreuna cu denumirea școlii de proveniență și județul.

```
CREATE OR REPLACE VIEW V_CANDIDATI_SCOLI AS
```

```
SELECT c.id_candidat,
```

```
       c.num_e_complet,
```

```
       c.cnp,
```

```
       c.id_unitate,
```

```
       u.denumire AS nume_scoala,
```

```
       u.id_judet
```

```
FROM candidat c
```

```
JOIN unitate_invatamant u ON c.id_unitate = u.id_unitate;
```

-- 2. Operație LMD Permisa

-- Enunț: Modificăm numele unui candidat prin intermediul vizualizării.

-- Acest UPDATE este permis pt ca modifica exclusiv un atribut dintr-un 'key-preserved table' (tabelul baza CANDIDAT) în cadrul vizualizării.

```
UPDATE V_CANDIDATI_SCOLI
```

```
SET nume_complet = 'Avram Mihai-Alexandru'
```

```
WHERE id_candidat = 80;
```

```
-- (Va afisa: 1 row updated)
```

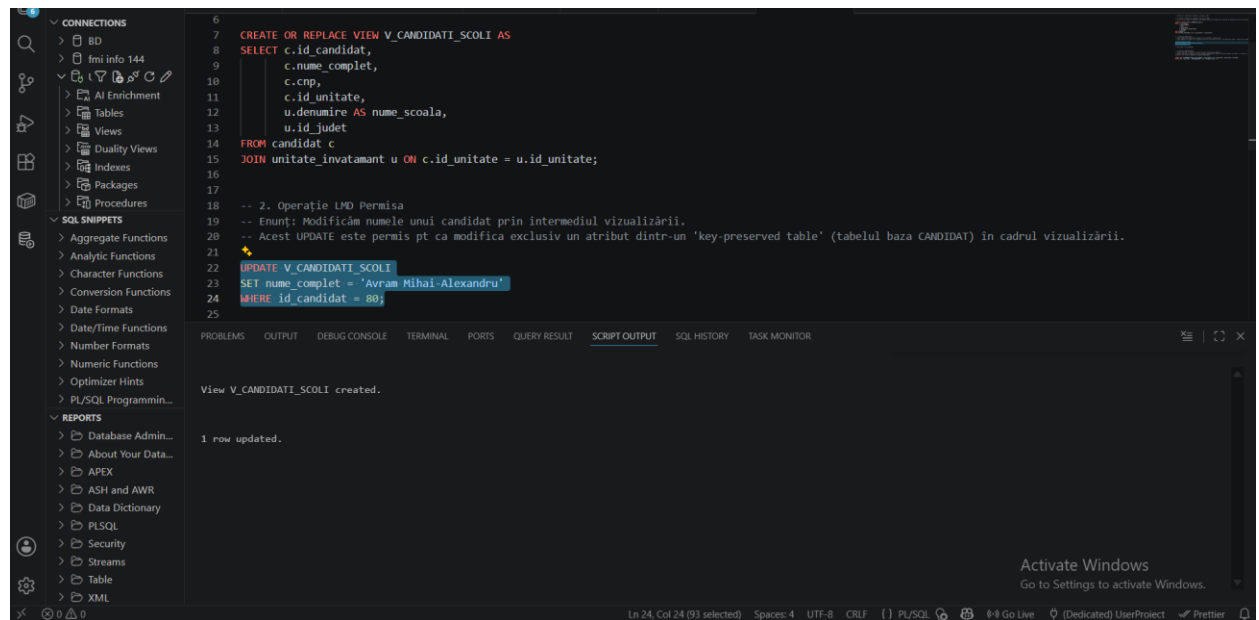
-- 3. Operație LMD Nepermisă

-- Enunț: Încercăm să inserăm o linie nouă prin vizualizarea complexă.

-- Această operație va genera o eroare, deoarece vizualizarea implică mai multe tabele (un JOIN), iar Oracle nu știe cum să împartă datele inserate între tabelul CANDIDAT și UNITATE_INVATAMANT.

```
INSERT INTO V_CANDIDATI_SCOLI (id_candidat, nume_complet, cnp, id_unitate, nume_scoala, id_judet)
```

```
VALUES (99, 'Test Elev', '5060708123456', 50, 'Colegiul Fals', 1);
```



15. Formulați în limbaj natural și implementați în SQL: o cerere ce utilizează operația *outer-join* pe minimum 4 tabele, o cerere ce utilizează operația *division* și o cerere care implementează analiza top-*n*.

-- Cerinta 15: 3 Cereri Suplimentare (Outer-Join, Division, Top-N)

-- 1. OUTER-JOIN pe minimum 4 tabele

-- Enunt: Sa se afiseze toti candidatii inscrisi, scoala de proveniență, codul secret al lucrarii și nota obținută.

-- Folosind LEFT OUTER JOIN, ne asigurăm că afișăm candidații chiar dacă aceștia nu au depus încă o lucrare sau nu au primit o notă.

```
SELECT c.num_e_complet,  
  
       u.denumire AS scoala_provenienta,  
  
       l.cod_secret,  
  
       e.nota_acordata  
  
FROM candidat c  
  
LEFT OUTER JOIN unitate_invatamant u ON c.id_unitate = u.id_unitate  
  
LEFT OUTER JOIN lucrare l ON c.id_candidat = l.id_candidat  
  
LEFT OUTER JOIN evaluare e ON l.id_lucrare = e.id_lucrare  
  
ORDER BY c.num_e_complet;
```

-- 2. Operația DIVISION (Diviziune)

-- Enunt: Să se gasească numele candidaților care au susținut și au depus lucrări la ABSOLUT TOATE sesiunile de examen disponibile în sistem (rezolvare clasică prin dublu NOT EXISTS).

-- Notă: În setul nostru curent de date de test, niciun elev nu a participat chiar la toate cele 5 probe posibile, deci rezultatul corect afișat va fi un tabel gol (ceea ce validează filtrarea corectă).

```
SELECT c.num_e_complet  
  
FROM candidat c  
  
WHERE NOT EXISTS (
```

```

SELECT 1

FROM sesiune s

WHERE NOT EXISTS (

    SELECT 1

    FROM lucrare l

    WHERE l.id_candidat = c.id_candidat

    AND l.id_sesiune = s.id_sesiune

)

);

```

-- 3. Analiza TOP-N

-- Enunt: Să se afișeze Top 3 cele mai bune lucrări din întregul județ, incluzând numele candidatului și nota obținută, ordonate descrescător după notă.

```

SELECT nume_complet, nota_acordata

FROM (

    SELECT c.nume_complet, e.nota_acordata

    FROM candidat c

    JOIN lucrare l ON c.id_candidat = l.id_candidat

    JOIN evaluare e ON l.id_lucrare = e.id_lucrare

    ORDER BY e.nota_acordata DESC

)

WHERE ROWNUM <= 3;

```

1 -- Cerinta 15: 3 Cereri Suplimentare (Outer-Join, Division, Top-N)
2
3 -- 1. OUTER-JOIN pe minimum 4 tabele
4 -- Enunt: Sa se afiseze toti candidatii inscrisi, scoala de provenienta, codul secret al lucrarii si nota obtinuta.
5 -- Folosind LEFT OUTER JOIN, ne asiguram ca afisam candidatii chiar daca acestia nu au depus inca o lucrare sau nu au primit o nota.
6
7
8 SELECT c.numc_complet,
9 u.denumire AS scoala_provenienta,
10 l.cod_secret,
11 e.nota_acordata
12 FROM candidat c
13 LEFT OUTER JOIN unitate_invatamant u ON c.id_unitate = u.id_unitate
14 LEFT OUTER JOIN lucrare l ON c.id_candidat = l.id_candidat
15 LEFT OUTER JOIN evaluare e ON l.id_lucrare = e.id_lucrare
16 ORDER BY c.numc_complet;
17
18 -- 2. Operatia DIVISION (Diviziune)
19
20 -- Enunt: Sa se gaseasca numele candidatilor care au sustinut si au depus lucrari la ABSOLUT TOATE sesiunile de examen disponibile in sistem

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MONITOR

All rows fetched: 9 in 0.173 seconds

	NUMC_COMPLET	SCOALA_PROVENIENTA	COD_SECRET	NOTA_ACORDATA
1	Avram Mihai	Colegiul National Sf. Sava	SEC111	10
2	Avram Mihai	Colegiul National Sf. Sava	SEC111	10
3	Avram Mihai	Colegiul National Sf. Sava	SEC222	9
4	Badea Alina	Colegiul National Sf. Sava	SEC333	10
5	Badea Alina	Colegiul National Sf. Sava	SEC333	10
6	Cretu George	Liceul Emil Racovita	SEC444	7.5
7	Cretu George	Liceul Emil Racovita	SEC444	7.5
8	Dima Florin	Colegiul Banatean	SEC555	6
9	Ene Larisa	Liceul National Iasi	SEC666	8.2

Ln 7, Col 1 (346 selected) Spaces: 4 UTF-8 CRLF {} PL/SQL 8-8 Go Live (Dedicated) UserProject Prettier

38 -- Enunt: Sa se afiseze Top 3 cele mai bune lucrari din intregul judet, incluzand numele candidatului si nota obtinuta, ordonate descrescator dupa
39 nota.
40
41 SELECT numc_complet, nota_acordata
42 FROM (
43 SELECT c.numc_complet, e.nota_acordata
44 FROM candidat c
45 JOIN lucrare l ON c.id_candidat = l.id_candidat
46 JOIN evaluare e ON l.id_lucrare = e.id_lucrare
47 ORDER BY e.nota_acordata DESC
48)
49 WHERE ROWNUM <= 3;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MONITOR

All rows fetched: 3 in 0.094 seconds

	NUMC_COMPLET	NOTA_ACORDATA
1	Avram Mihai	10
2	Badea Alina	10
3	Avram Mihai	10

Ln 40, Col 1 (270 selected) Spaces: 4 UTF-8 CRLF {} PL/SQL 8-8 Go Live (Dedicated) UserProject Prettier

17. a) Realizarea normalizării BCNF, FN4, FN5

Pentru a demonstra aceste forme normale avansate, vom extinde schema cu relații ipotetice care încalcă regulile, urmate de descompunerea lor corectă.

1. Forma Normală Boyce-Codd (BCNF)

Definiție: O relație R se află în BCNF dacă și numai dacă, pentru orice dependență funcțională nontrivială $X \rightarrow Y$ din R, X este supercheie a lui R.

Exemplu de incalcare BCNF:

Extindem schema cu o relație care gestionează profesorii coordonatori de săli:

COORDONATOR_SALA(id_profesor, id_sala, tip_subiect)

Reguli de business:

- Un profesor coordonează exact o singură sală: $\text{id_profesor} \rightarrow \text{id_sala}$ (dar într-o sală pot fi mai mulți coordonatori, pentru subiecte diferite).
- În fiecare sală există cel mult un coordonator per tip de subiect: $(\text{id_sala}, \text{tip_subiect}) \rightarrow \text{id_profesor}$.

Chei candidate:

- CC1 = {id_profesor, tip_subiect}
- CC2 = {id_sala, tip_subiect}

Verificare BCNF: Dependența $\text{id_profesor} \rightarrow \text{id_sala}$ există, dar id_profesor singur nu este supercheie. Deci BCNF este incalcata.

Descompunere în BCNF:

- PROFESOR_SALA(id_profesor, id_sala) - având PK: id_profesor
- PROFESOR_SUBIECT(id_profesor, tip_subiect) - având PK: (id_profesor, tip_subiect)

2. A Patra Formă Normală (FN4)

Definiție: O relație R se află în FN4 dacă și numai dacă, pentru orice dependență multivalorică nontrivială $X \twoheadrightarrow Y$ din R, X este supercheie a lui R.

Exemplu de incalcare FN4: Să presupunem că un profesor poate avea mai multe specializări (Română, Engleză) și poate face parte din mai multe comisii județene (Evaluare, Contestații), complet independente unele de altele. PROFESOR_DETALII(id_profesor, specializare, comisie)

Dependențe multivalorice (MVD):

- $\text{id_profesor} \twoheadrightarrow \text{specializare}$
- $\text{id_profesor} \twoheadrightarrow \text{comisie}$

Aceste două MVD-uri sunt independente. Dacă un profesor primește o specializare nouă, ar trebui inserată câte o linie pentru fiecare comisie din care face deja parte, generând redundanță masivă și risc de anomalii de actualizare.

Descompunere în FN4:

- PROF_SPECIALIZARE(id_profesor, specializare) - PK: (id_profesor, specializare)
- PROF_COMISIE(id_profesor, comisie) - PK: (id_profesor, comisie)

3. A Cincea Formă Normală (FN5)

Definiție: O relație R se află în FN5 dacă și numai dacă orice dependență de reuniune (join dependency) nontrivială din R este implicată de cheile candidate ale lui R.

Exemplu de incalcare FN5:

Avem o relație care leagă profesorii evaluatori de centrele de examen și de discipline:

EVALUATOR_CENTRU_DISCIPLINA(id_profesor, id_centru, id_disciplina)

Regulă de business (constrângere ciclică): Un profesor evaluează la un centru la o disciplină dacă și numai dacă:

1. Profesorul este repartizat la acel centru.
2. Profesorul este certificat să predea acea disciplină.
3. Centrul respectiv organizează corectura pentru acea disciplină.

Cheia candidată este întreaga relație (id_profesor, id_centru, id_disciplina). Dacă relația conține toate cele trei proiecții binare, tripletul trebuie să existe. Dacă am insera date manual în acest tabel ternar, am risca inconsistențe logice. FN5 este incalcata.

Descompunere în FN5:

Se descompune în 3 proiecții binare care, prin JOIN, refac relația fără pierderi:

- PROF_CENTRU(id_profesor, id_centru)
- PROF_DISC(id_profesor, id_disciplina)
- CENTRU_DISC(id_centru, id_disciplina)

b) Aplicarea denormalizării, justificând necesitatea acesteia

Denormalizare propusă: Adăugarea atributului medie_generala direct în tabelul CANDIDAT.

Justificare: În modelul relațional actual, complet normalizat, pentru a afișa media generală a unui elev trebuie să realizăm operații de tip JOIN între tabelele CANDIDAT, LUCRARE și EVALUARE, urmate de o funcție de agregare AVG() aplicată pe note. În perioada publicării

rezultatelor examenelor naționale, mii de utilizatori accesează simultan sistemul pentru a genera topuri și rapoarte cu mediile finale. Calcularea on-the-fly a acestor medii prin JOIN-uri masive ar consuma extrem de multe resurse și ar încetini serverul. Prin denormalizare (stocarea mediei precalculate direct în entitatea CANDIDAT), se introduce o redundanță controlată. Sacrificăm puțin spațiu de stocare și adăugăm o penalizare minoră la scriere (media trebuie actualizată când se modifică o notă), dar câștigăm o viteză masivă la citire (operațiile de tip SELECT pentru topuri/ierarhii județene se vor face instantaneu dintr-un singur tabel).

19. Justificarea necesității/utilității migrării la o bază de date de tip NoSQL

Scenariul care avantajează migrarea: O bază de date NoSQL (de exemplu, una orientată pe documente, precum MongoDB) ar deveni extrem de avantajoasă dacă Ministerul Educației ar decide trecerea la lucrări de examen complet digitalizate.

Justificare:

1. Flexibilitatea schemei (Schema-less): Într-o bază de date relațională, structura datelor este rigidă. Dacă elevii ar depune fișiere diferite (cod sursă pentru proba de Informatică, scheme scanate pentru Fizică sau eseuri text pentru Română), stocarea lor eficientă într-un tabel LUCRARE ar fi greoaie. O bază de date NoSQL orientată pe documente ar permite stocarea profilului candidatului împreună cu absolut toate aceste metadate și fișiere de formate diferite într-un singur document JSON.
2. Scalabilitate orizontală nativă: Sistemele de examene naționale se confruntă cu vârfuri uriașe de trafic (spike-uri) în cele 2-3 zile de afișare a rezultatelor. Spre deosebire de bazele relaționale (cum este Oracle), care scalează predominant vertical (necesită servere fizice mai puternice și scumpe), bazele NoSQL sunt construite să scaleze orizontal foarte ușor, distribuind încărcarea pe zeci de noduri/servere mai ieftine pentru a face față cererilor masive, prevenind astfel blocarea sistemului.

// a) Prezentarea structurii bazei de date NoSQL (JSON)

// Intr-o baza NoSQL, datele candidatului, scoala si notele pot sta in acelasi document.

{

"candidatId": 80,

"nume_complet": "Avram Mihai",

"scoala": { "denumire": "Colegiul National Sf. Sava", "judet": "Bucuresti" },

```
"lucrari": [  
  { "disciplina": "Limba Romana", "nota": 9.50 },  
  { "disciplina": "Matematica", "nota": 9.00 }  
]  
}
```

```
// b) Comenzi pentru crearea bazei de date (sau selectarea ei)  
use('examene_nationale_db');
```

```
// c) Comenzi pentru inserare, modificare și ștergere (CRUD)
```

```
// INSERARE
```

```
db.candidati.insertMany([  
  {  
    "candidatId": 80,  
    "nume_complet": "Avram Mihai",  
    "scoala": { "denumire": "Colegiul National Sf. Sava", "judet": "Bucuresti" },  
    "lucrari": [ { "disciplina": "Limba Romana", "nota": 9.50 } ]  
  }  
]);
```

```
// MODIFICARE (Actualizam nota)
```

```
db.candidati.updateOne(  
  { "candidatId": 80, "lucrari.disciplina": "Limba Romana" },
```

```
    { $set: { "lucrari.$.nota": 9.80 } }  
  );
```

```
// STERGERE
```

```
db.candidati.deleteOne({ "candidatId": 80 });
```

```
// d) Exemplificarea comenzilor pentru interogare (Filtrare și sortare)
```

```
// Gasim candidatii din Bucuresti cu nota peste 9 la Romana si ii sortam alfabetic
```

```
db.candidati.find({  
  "scoala.judet": "Bucuresti",  
  "lucrari": { $elemMatch: { "disciplina": "Limba Romana", "nota": { $gte: 9.00 } } }  
}).sort({ "nume_complet": 1 });
```